Deakin University
Faculty of Science, Engineering and Built Environment
School of IT

# Assignment 1 Report

SIT215 – Computational Intelligence

Trimester 1, 2026

Student Name: Bao Minh Tran
Student ID: s224236373

| Tasks Completed | Completed (✓) | Notes (Optional) |
|---|---|---|
| Task 1: Environment modelling and problem formulation | ☐ | |
| Task 2: Search-based agent implementation and comparison<br>• $A^*$<br>• Selected uninformed algorithm | ☐ | |
| Task 3: Success metrics and evaluation | ☐ | |
| Task 4: New heuristic design and performance | ☐ | |

## Contents

**1 Problem Formulation** **3**
1.1 Problem Context . . . 3
1.2 Problem Constraints and Environment Selection Reasoning . . . 3

**2 Modelling Approach** **4**
2.1 PEAS Framework . . . 4
2.2 Environment Abstraction into State Space: A relaxed version . . . 4
2.2.1 Cost Model . . . 5
2.2.2 State Space Representation . . . 6

**3 Search Strategies and Heuristic Design** **6**
3.1 Benchmark Algorithm: A* . . . 6
3.2 Baseline Heuristic: Manhattan Distance . . . 6
3.3 Selected Uninformed Search Algorithm . . . 6
3.4 Comparison Rationale . . . 6
3.5 Heuristic Properties . . . 6
3.6 New Heuristic Design . . . 6
3.7 Expected Impact of the New Heuristic . . . 6

**4 Implementation Overview** **7**
4.1 Overall System Structure . . . 7
4.2 Data Structures . . . 7
4.3 Node Expansion Logic . . . 7
4.4 Constraint Validation Logic . . . 7
4.5 Implementation of A* . . . 7
4.6 Implementation of the Uninformed Search Algorithm . . . 7
4.7 Implementation of the New Heuristic . . . 7
4.8 Code Snippet References . . . 7

**5 Results and Evaluations** **8**
5.1 Experimental Setup . . . 8
5.2 Test Cases . . . 8
5.3 Performance Metrics . . . 8
5.4 Comparison of A* and Uninformed Search . . . 8
5.5 Evaluation of Intelligence Quality . . . 8
5.6 Comparison of Heuristics . . . 8
5.7 Visualisations . . . 9
5.8 Discussion . . . 9

**6 Conclusion** **10**

# 1. Problem Formulation

## 1.1 Problem Context

Vingroup JSC [1] is a multi–sector corporation that has thrived as one of the top conglomerates in Vietnam. Since 2000s, it has expanded across different domains, e.g. VinFast - multinational automotive manufacturer, VinAI - leading AI Research company, and many more. It is now aiming to open a new subsidiary, namely VinExpress, focusing on courrier, express, and parcels service market, which is a profitable sector, but has been predominated [2] by ViettelPost, VietnamPost, and J&T Express.

Prior to expanding the market nationally and potentially on a global scale, it first demands a comprehensive efficient route planning in Ho Chi Minh City (HCMC), the largest metropolitan city in Vietnam. Therefore, this report focuses on planning in a small part of the city to evaluate the feasibility and competitiveness when scaling to the whole HCMC. The map is given below.

Figure 1: A snapshot of a selected region of HCMC with 21 landmarks adapted from GoogleMaps [3].

## 1.2 Problem Constraints and Environment Selection Reasoning

As shown in the map, it is a densely populated residential area, which has imposed pressure on transport infrastructure and led to serious traffic congestion and poorly maintained streets [4]. Express delivery companies must propose appropriate measures to efficiently allocate budget for transportation and management while protecting fragile parcels and on–time delivery. Moreover, planners must be aware of vehicle restrictions in specific roads, traffic flow at different time periods, one–way roads, and limited road widths. Under this context, route feasibility is attributed to two **dynamic constraints**, i.e. vehicle restrictions and traffic flow, and several **static constraints** relating to roads, e.g. distances, directions and road conditions.

As depicted above, the 21 selected landmarks are subject to spread across the map, so that some are near to the avenues, whereas some are located within inner and smaller streets. Besides, distances between landmarks vary considerably. Each path segment is assigned to a weight, which represents projected petrol costs considering traffic flow and distance. This is more reasonable and realistic than using distance, as time taken to travel in some roads may be more or less than expected due to traffic

situations. Fortunately, road fees can be neglected as there are no toll stations in the central of HCMC.

## 2. Modelling Approach

### 2.1 PEAS Framework

The problem statement above can be summarised more concisely by the utilisation of PEAS framework [5] to formally define the rationality standards [6] of truck delivery agent.

**Environment (E)**

- Partially observable (blind spots of truck)
- Multi–agent (other vehicles)
- Nondeterministic (accidents)
- Sequential (braking may cause late delivery)
- Dynamic (traffic flows vary)
- Continuous (time and actions, e.g. speed, rotatation are continuous)
- Known (truck must follow the designated routes)

**Performance (P)**

- Fuel cost optimisation
- Timely delivery
- No items are broken during transportation
- Right items, right places

**Sensors (S)**

- Traffic observations
- Clock
- GPS location
- Delivery status

**Actuators (A)**

- Accelerate
- Brake
- Steering

### 2.2 Environment Abstraction into State Space: A relaxed version

Having studied the PEAS framework above, it is apparent that the problem context is far from simplicity. Although it reflects the real–world scenarios well, it is arduous to implement an effective and efficient agent in such environment. Therefore, in this work, we would relax the problem by maintaing the static constraints while reducing dynamic constraints into static constraints. Additionally, to closely reflect different costs during transportation, we establish a weighted graph, which impose different weights to each path connecting two landmarks.

As opposed to Figure 1, where there is a plenty of roads, from avenues to minor roads connecting the destinations. In the relaxed problem, they are limited to commonly operated streets. Thereby, graph, a discrete data structure, is utilised to represent the problem, as it allows us to introduce states of the environment better.

To preserve the static constraints of the original problem, i.e. road directions, we leverage a **directed graph** [7], which allows the agent to move in specific directions. More importantly, to ensure the feasibility of the problem [5], this directed graph must be strongly connected [7]. Another static constraint is road condition, as streets are rarely fixed due to various reasons. Indeed, these poorly maintained

roads may deteriorate the truck and some fragile parcels may be broken while being transported on such roads. To reflect the degree of conditions, we leverage a **risk factor**, denoted as $\tau(u, v) \in (0, 1)$, indicates the likelihood of transported items may be broken on the path from $u$ to $v$, assumming that we could gather relevant data. The last static constraint is about distance between two landmarks $u$ and $v$, denoted as $d(u, v)$, in this problem, we utilised GoogleMaps [3] to collect these numbers.

Although only the static constraints are enfored in the relaxed problem, we would like to somewhat mimic the dynamic constraints at some extent, i.e. traffic flow. This factor greatly impacts the travelling time and unintended adverse accidents. Therefore, we introduce the utilisation a traffic congestion coefficient, denoted as $\psi(u, v)$, which partly describe how bad the traffic is on specific roads. Regarding delivery time, we leverage the estimated time, denoted as $t(u, v)$, adapted from GoogleMaps [3]. Another dynamic constraint is restricted access zones in certain time periods. However, this is quite hard to reflect, thus we ignore this constraint, and only represent one– and two–ways streets.

### 2.2.1 Cost Model

In this section, we present the mathematical model that is used to reasonably assign the costs to different path segments connecting the landmarks. This cost model incorporates several functions that strictly follows the interpretations above. Remarkably, the information of this model is not given to the agent by any means.

Given a graph $G = (V, E)$, for an edge $(u, v) \in E$. The corresponding weight, denoted as $\Omega(u, v)$, given by:

$$\omega(u, v) = \alpha \cdot d(u, v) + \beta \cdot t(u, v) \tag{1}$$

$$\Omega(u, v) = (1 + \tau(u, v) + \psi(u, v)) \times \omega(u, v) \tag{2}$$

where:

- $d(u, v)$: estimated distance (in km) adapted from GoogleMaps [3]
- $\alpha$: fuel comsumption cost, in our context, $\alpha = 2.3$
- $t(u, v)$: estimated time (in hour) adapted from GoogleMaps [3]
- $\beta$: time contribution coefficient, in our context, $\beta = 2 \times \alpha = 4.6$
- $\tau(u, v) \in (0, 1)$: risk factor
- $\psi(u, v) \in (0, 1)$: traffic congestion coefficient

Beyond the aforementioned factors contributing to the cost connecting two landmarks, we introduce two auxiliary coefficients, i.e. $\alpha$ and $\beta$, which demonstrates our focus when searching for an optimal path, which are fuel consumption and delivery time. Noticeably, we let $\beta > \alpha$ as we would like to emphasize the **timely delivery** mission of VinExpress. In other words, we may prioritise routes that are a little further but fewer vehicles, and thereby move faster, over those are less distant but overly crowded.

Briefly speaking, this cost model reflects our concerns when finding the optimal path, by putting into consideration several factors and aggreagting them by addition and multiplication. Firsly, $\omega(u, v)$ considers time and distance, which are two important factors in our problem, and allocates different coefficients, $\alpha$ and $\beta$ to specifies our priority. Secondly, we amplify this path cost $\omega(u, v)$ by considering traffic congestion degree $\psi(u, v)$ and the road conditions $\tau(u, v)$. Lastly, we add up these amplified costs with our pure one, to arrive at the final weight, i.e. $\Omega(u, v)$.

#### 2.2.2 State Space Representation

The below graph represents the relaxed problem, which includes nodes, numbered from 1 to 21, akin to those in Figure 1. Additionally, the weights are assigned to edges connecting pairs of nodes, which were computed accordingly to the proposed **Cost Model** above. Noticably, the weights were rounded for the ease of visualisation.

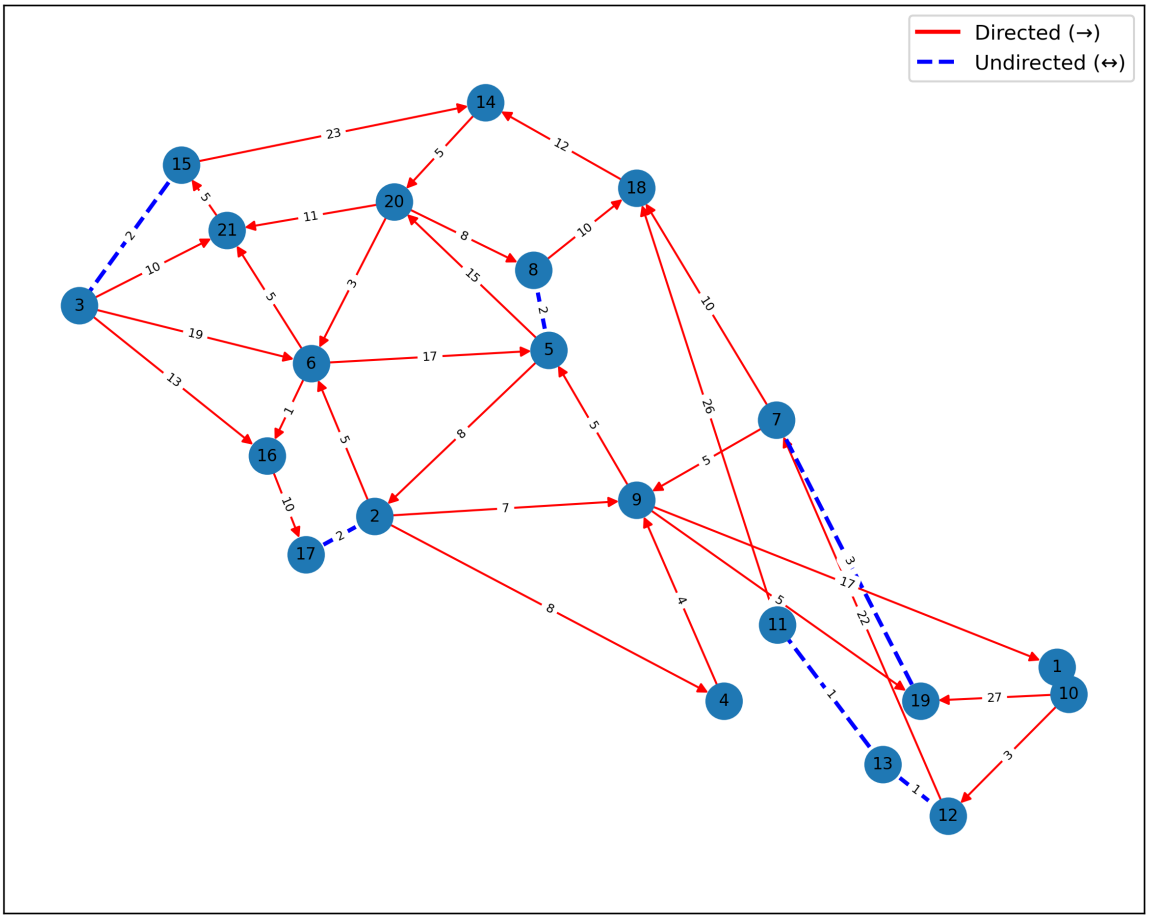


Figure 2: A state space representation of the relaxed problem

## 3. Search Strategies and Heuristic Design

### 3.1 Benchmark Algorithm: A*

### 3.2 Baseline Heuristic: Manhattan Distance

### 3.3 Selected Uninformed Search Algorithm

### 3.4 Comparison Rationale

### 3.5 Heuristic Properties

### 3.6 New Heuristic Design

### 3.7 Expected Impact of the New Heuristic

# 4. Implementation Overview

## 4.1 Overall System Structure

## 4.2 Data Structures

## 4.3 Node Expansion Logic

## 4.4 Constraint Validation Logic

## 4.5 Implementation of A*

## 4.6 Implementation of the Uninformed Search Algorithm

## 4.7 Implementation of the New Heuristic

## 4.8 Code Snippet References

Listing 1: Placeholder for annotated code snippet with line numbers.

```
# Paste relevant code snippet here
# Make sure line numbers are visible in the PDF output
```

Listing 2: A* Search Algorithm

```
def astar(graph, start, goal, heuristic):
    open_set = PriorityQueue()
    open_set.put((0, start))

    came_from = {}
    g_score = {start: 0}

    while not open_set.empty():
        _, current = open_set.get()

        if current == goal:
            return reconstruct_path(came_from, current)

        for neighbor, cost in graph[current]:
            tentative = g_score[current] + cost

            if neighbor not in g_score or tentative < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative
                f_score = tentative + heuristic(neighbor, goal)
                open_set.put((f_score, neighbor))
```

# 5. Results and Evaluations

## 5.1 Experimental Setup

## 5.2 Test Cases

| Case | Start Node | Goal Node | Notes |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

Table 1: Start-goal test cases used for evaluation.

## 5.3 Performance Metrics

## 5.4 Comparison of A* and Uninformed Search

| Case | Algorithm | Path Found | Path Cost | Nodes Expanded | Runtime | Optimal? |
|---|---|---|---|---|---|---|
| 1 | A* | | | | | |
| 1 | Uninformed | | | | | |
| 2 | A* | | | | | |
| 2 | Uninformed | | | | | |
| 3 | A* | | | | | |
| 3 | Uninformed | | | | | |

Table 2: Performance comparison of A* and the selected uninformed search algorithm.

## 5.5 Evaluation of Intelligence Quality

## 5.6 Comparison of Heuristics

| Case | Heuristic | Path Cost | Nodes Expanded | Runtime | Comments |
|---|---|---|---|---|---|
| 1 | Manhattan | | | | |
| 1 | New heuristic | | | | |
| 2 | Manhattan | | | | |
| 2 | New heuristic | | | | |
| 3 | Manhattan | | | | |
| 3 | New heuristic | | | | |

Table 3: Comparison between Manhattan distance and the new heuristic.

### 5.7 Visualisations

Figure 3: Placeholder for result visualisation (e.g., runtime or node expansion comparison).

### 5.8 Discussion

## 6. Conclusion

## Acknowledgement

## References

[1] Vingroup, "About vingroup," n.d. Accessed: 2026-03-18.

[2] VietnamCredit, "Top 5 courier companies in vietnam," 2020. Accessed: 2026-03-18.

[3] Google, "Google maps," 2026. Accessed: 2026-03-18.

[4] United Nations Economic Commission for Europe, "Road safety performance review: Viet nam," 2018. Accessed: 2026-03-18.

[5] Deakin University, "Sit215: Computational intelligence." Unit materials, Deakin University, 2026. Accessed: 2026-03-18.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson Education Limited, 4 ed., 2021. Global Edition, Accessed: 2026-03-18.

[7] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*. Wiley, 6 ed., 2014.